SATELLITE EMBEDDED SYSTEMS / 01

Satellite Switch Bare-Metal Port 95% RAM Cut

I owned the Linux-to-bare-metal port of the MESA Ethernet-switch SDK for a satellite project: ARM build integration, MCU platform abstractions, allocator instrumentation, RAM optimization, and runtime debugging.

MY MEASURED OPTIMIZATION

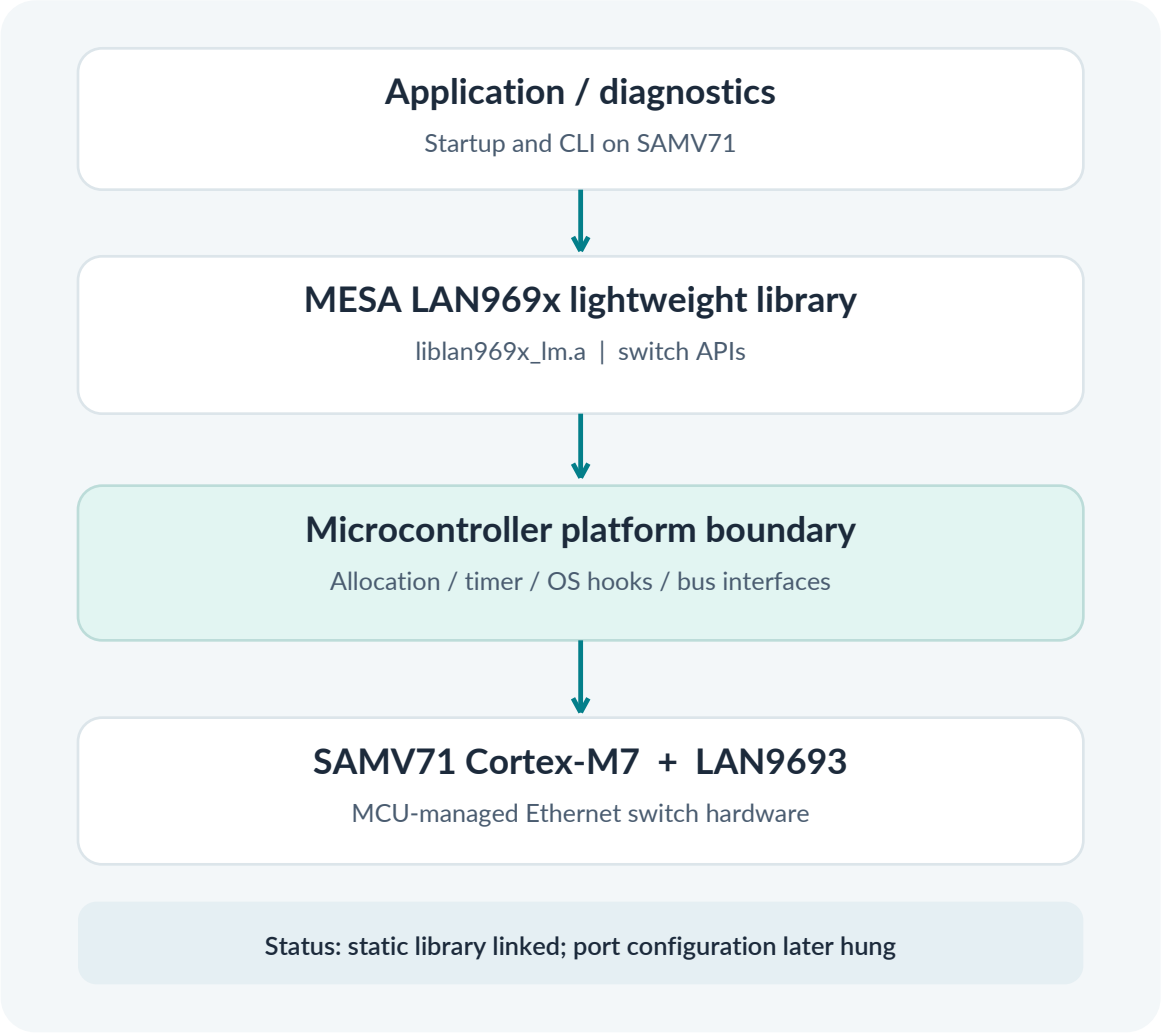
3.87 MB > 193 KB

Main runtime heap allocation reduced from 3,870,304 B to 193,376 B (approximately 95%).

SAMV71 Cortex-M7 / LAN9693 Ethernet switch

Static library linked; full switch bring-up not completed

SYSTEM OVERVIEW / LOGICAL, NOT A PCB SCHEMATIC



MY SCOPE: PORT + PROFILING + DEBUGGING / NOT VERIFIED FORWARDING

02 / MY IMPLEMENTATION

Porting MESA from Linux to bare-metal ARM

I owned the end-to-end porting effort for the selected SDK target: adapting Linux-dependent platform interfaces, configuring cross-compilation, linking the library into SAMV71 firmware, and debugging execution on the MCU.

BUILD CONFIGURATION / SOURCE: CMAKE + TOOLCHAIN

```
MESA_OS_CUSTOM = ON
TARGET       = lan969x_lm
OPT_LIGHT    = 1    PORT_COUNT = 24
arm-none-eabi-gcc  -mcpu=cortex-m7
```

THUMB | SOFTFP ABI | FPU: FPV4-SP-D16

CUSTOM OS BOUNDARY / EXCERPT FROM MODIFIED HEADER

```
#define VTSS_OS_MALLOC(n, f)  mesa_malloc(n)
#define VTSS_OS_FREE(p, f)    mesa_free(p)
#define VTSS_MSLEEP(ms)      delay_ms(ms)
/* Timer ticks via get_global_ms() */
```

SOURCE: include/vtss_os_custom.h | simplified identifiers

MY SPECIFIC ENGINEERING CONTRIBUTIONS

Ported OS hooks

I mapped MESA allocation and timing dependencies to bare-metal MCU functions.

Built + linked

I configured the ARM toolchain, compiled MESA and linked its static library into firmware.

Debugged runtime

I traced execution into port mapping and observed the debugger hang and CLI freeze.

MY VALIDATION

I compiled and linked MESA, reached `mesa_port_map_set()` on the MCU, and traced a hang in the debugger; the CLI froze and the root cause remained unresolved.

03 / MY MEMORY PROFILING

How I reduced runtime state RAM by 95%

I instrumented `vtss_state_t` and `mesa_malloc()`, profiled oversized tables, and remeasured after each trim.

ONE PERSISTENT RUNTIME HEAP ALLOCATION / NOT FLASH OR FULL-PROGRAM RAM

3,870,304 B → **193,376 B**

95.0% SMALLER

MY MEASURED HEAP REQUESTS / NINE BUILD CHECKPOINTS

Full configuration	**3,870,304**
VLAN capacity: 4,096 to 64	3,128,416
MAC capacity: 16,384 to 128	2,672,736
Early 24-port trim	2,612,736
Lightweight MESA	291,264
Light + reduced VLAN	250,944
TAS-off snapshot (30 ports)	240,384
Revised 24-port build	222,496
Final L3 / VCAP trimmed	**193,376**

WHY THE HEAP ALLOCATION SHRANK

MESA allocates one large vtss_state_t object. Embedded arrays grow with compile-time limits, even when their full capacity is unused.

```
vlan_table[VTSS_VIDS];
mac_table[VTSS_MAC_ADDRS];
```

ORIGINAL STATE COMPONENTS / MEASURED

Layer 2: 2.25 MB VCAP: 0.73 MB

Layer 3: 0.51 MB Timestamp: 10 KB

Final allocation: 193,376 B / 188.8 KiB

Requests are per-build runtime allocations, not full-program RAM. Some intermediate port configurations differ; the final size is verified in the log.

Evidence: full nine-stage profiling log (click)

04 / RESULTS AND ENGINEERING DECISION

Why the MCU architecture was reconsidered

My profiling showed that large runtime state reductions were possible, but the remaining MESA heap plus static globals still consumed most of the available SRAM. The port reached initialization, not completed switch bring-up.

APPROXIMATE SRAM DEMAND / INDEPENDENT HEAP AND STATIC ALLOCATIONS

384 KiB total SRAM

~75% accounted for by MESA-related storage



The ~100 KB covers separate MESA `.bss` / `.data` globals (not the heap allocation). Remaining RAM also has to accommodate stack, application state, other allocations, and headroom.

WHAT I PERSONALLY VERIFIED

1. **Built and linked**
   I integrated the trimmed ARM static library into SAMV71 firmware.
2. **Reached initialization**
   I traced execution into mesa_port_map_set().
3. **Debugged the hang**
   I observed the stalled debugger and frozen CLI; cause not confirmed.

TEAM-LEVEL ARCHITECTURE DECISION

My porting work and profiling quantified the MCU constraints. With ~193 KB of heap state and ~100 KB of separate MESA globals, senior engineers decided to explore Linux-board alternatives rather than continue the constrained MCU approach.

I transitioned to another assignment after that decision.

ENGINEERING TAKEAWAY — **Making one allocation fit is not the same as proving the whole architecture viable.**